

第 2 周：AI 辅助编程工具实战

怎么让 AI 帮我写代码？

北京石油化工学院 人工智能研究院 王文通

通选课

2025-2026 学年

需要的预备知识:

- Python 基础

- 函数定义与调用
- 循环与条件判断
- 列表和字典操作

- OpenCV 基础

- 图像读取与显示
- 基本的图像变换

预备资源:

- Python 基础

- <https://www.bilibili.com/video/BV1Jgf6YvE8e>

- OpenCV 入门

- <https://www.bilibili.com/video/BV1iF411T7zj>

重要提示

本课重点是学习如何与 AI 协作编程，不要求深入的编程基础！

本周内容:

- LLM 时代的编程新范式
- AI 编程工具全景
- Prompt 工程精讲
- CV 领域深度实战
- 调试与重构

核心技能:

- 结构化 Prompt 设计 (RTF)
- 思维链 (Chain-of-Thought)
- 少样本提示 (Few-shot)
- 多轮迭代优化
- 代码审查与验证

本周时间分配 (3 学时)

第 1 学时 (50 分钟):

- 00:15 理论: AI 编程范式 (15min)
- 00:30 演示: AI 工具使用 (15min)
- 00:50 实践: Prompt 设计练习 (20min)

第 2 学时 (50 分钟):

- 01:20 理论: Prompt 工程精讲 (30min)
- 01:40 实践: CV 代码实战 (20min)

第 3 学时 (60 分钟):

- 01:40-02:00 互动: Prompt Quiz (20min)
- 02:00-02:25 互动: 代码找错挑战 (25min)
- 02:25-02:40 总结与作业 (15min)

时间控制提示

如果进度落后, 可跳过代码找错挑战

本周分组策略

分组原则：

- 每 4 人为一组
- 不同专业背景混合
- 确保每组至少有一名有编程基础的同学

角色分工：

角色	职责	适合
组长	协调进度，分配任务	组织能力强的
技术负责人	把关代码质量，审核 AI 生成代码	有编程基础的
Prompt 设计师	负责设计高质量 Prompt	思维活跃的
测试者	负责代码测试与优化	细心负责的

本周协作任务

用 AI 辅助实现 答题卡边界检测

回顾：第 1 周的挑战

还记得这些痛点吗？

- cv2.imread() 返回 None，找不到原因
- 图像显示颜色异常
- 数组越界、维度不匹配
- API 参数太多，记不住

传统解决方式：

- ① 翻文档 (docs.opencv.org)
- ② 搜索 StackOverflow
- ③ 问同学/老师
- ④ 试错（耗费大量时间）

典型错误代码

```
# 死循环！
while img is not None:
    cv2.imshow('image', img)
    if cv2.waitKey(1) == 'q':
        break
```

AI 辅助的新方式

- 直接问 AI
- 解释错误原因
- 给出修改建议
- 降低学习门槛！

Token 预测

+

模式匹配

为什么 AI 擅长语法但不一定懂逻辑？

- **擅长：**基于海量代码库的模式复刻
- **不擅长：**理解你的具体业务逻辑

什么是 AI 幻觉?

AI 自信满满地生成看似合理但实际上错误或不存在的信息。

案例：AI 发明了一个不存在的 OpenCV 函数

AI 的“创意”代码

```
import cv2

img = cv2.imread('exam.jpg')

# 不存在的函数!
fixed = cv2.auto_fix_exposure(img)
```

如何识别幻觉？

识别方法

- 查官方文档 (docs.opencv.org)
- 在 Python 中 `dir(cv2)` 查看所有可用函数
- 多问一句：“这个函数真的存在吗？”

应对策略

- **先验证再使用**：运行代码检查是否有效
- **分段确认**：让 AI 解释代码每一步的作用
- **要求引用**：让 AI 提供函数出处的链接

编程范式的演进

1.0 阶段：查阅文档/StackOverflow

- 手动搜索
- 阅读文档
- 试错调试

效率：两星

2.0 阶段：AI 辅助

- 从“写作者”转变为“审查者”
- AI 生成初稿，人审查修改
- 快速原型验证

效率：四星

3.0 阶段：AI 原生？

- AI 直接理解需求生成完整系统
- 人主要负责需求定义和架构设计
- 可能的未来方向

效率：五星？

核心结论：程序员的核心竞争力转变

从“记忆语法”转向“问题定义”与“代码审查”

AI 编程工具演进路线



网页版工具

- **代表：**通义千问
- 无需安装
- 对话式交互
- 适合学习理解

IDE 集成工具

- **代表：**Cursor
- 无缝编码体验
- 实时辅助
- 适合日常开发

CLI 工具

- **代表：**Claude Code
- 命令行原生
- 项目级理解
- 适合高级用户

通义千问 = 国内领先的 AI 助手

<https://qianwen.com>

核心优势：

- **国内访问稳定**：无需翻墙，速度快
- **中文理解优秀**：专为中文优化
- **代码能力强**：Qwen-Coder 模型
- **完全免费**：基础功能免费使用
- **多功能集成**：支持文档、图片、代码

适合场景：

- 快速提问和学习
- 代码逻辑理解
- 中文文档生成
- 算法思路讨论
- API 查询和示例

访问方式

浏览器打开 qianwen.com，注册登录即可使用

通义千问实战演示

场景：快速理解 API 用法

- 1 打开 `qianwen.com`
- 2 输入问题
- 3 AI 给出解释和示例

技巧

明确描述问题、请求代码示例

Cursor = VS Code + AI

<https://cursor.sh>

核心优势：AI 原生设计、VS Code 兼容、上下文理解

适合场景：日常开发、代码重构、Bug 修复

安装方式

下载 `cursor.sh`, 安装后即可使用

Ctrl+K
原地编辑

Ctrl+K：原地编辑

- 选中代码
- 按 Ctrl+K
- 输入修改指令
- AI 原地修改

Ctrl+L
上下文对话

Ctrl+L：上下文对话

- 按 Ctrl+L
- 打开聊天面板
- 多轮对话
- 引用文件上下文

Ctrl+I
智能提示

@ 引用功能

- @Files：引用文件
- @Code：引用代码
- @Docs：引用文档
- @Web：搜索网络

Cursor 实战演示

原地编辑: 选中代码 → Ctrl+K → 输入指令 → AI 修改

技巧: 用 Ctrl+L 多轮对话、@Files 引用文件

Claude Code

<https://claude.ai/code>

优势: 命令行原生、项目级理解、多文件操作

场景: 大型项目重构、自动化脚本

安装: `npm install -g @anthropic-ai/claude-code`

Claude Code 核心命令

命令	功能
<code>claude</code>	启动交互式对话
<code>claude " 问题 "</code>	单次提问
<code>claude --file xxx</code>	指定文件上下文
<code>claude --edit</code>	编辑模式

使用流程： 进入目录 → 运行 `claude` → 描述需求 → 确认修改

MiniMax = 国产高性能大模型

<https://www.minimaxi.com>

核心优势：

- **国产自研**：完全自主可控
- **多模态能力**：支持文本、图像、语音
- **长上下文**：支持超长文本处理
- **性价比高**：价格实惠
- **API 友好**：完善的开发者接口

适合场景：

- 企业级应用开发
- 多模态任务
- 长文档处理
- API 集成开发
- 隐私敏感场景

使用方式

- 网页版：直接对话
- API：集成到应用中

三种工具对比

维度	网页版	Cursor	CLI
界面	浏览器	VS Code	命令行
曲线	极低	低	中
集成	无	深度	命令行
上下文	单次	当前文件	整个项目
实时性	需切换	实时辅助	命令触发
适合	所有用户	开发者	高级用户
最佳场景	学习/咨询	日常开发	项目重构

工具选择建议

根据场景选择

- **初学者**: 从网页版 (通义千问) 开始
- **日常开发**: 使用 Cursor, 享受实时 AI 辅助
- **大型项目**: 结合 CLI 工具 (Claude Code)

学习路径建议

- ① 第 1 周: 使用网页版工具熟悉 AI 对话
- ② 第 2 周: 学习 Cursor 的基本操作
- ③ 第 3 周后: 尝试 CLI 工具进行项目级操作

如何选择合适的工具?

场景 1: 学习新 API

- **推荐:** 通义千问
- 快速提问, 获得解释
- 中文理解优秀
- 无需安装, 即开即用

场景 3: 项目重构

- **推荐:** Claude Code
- 项目级理解
- 多文件操作
- 自动化批量处理

场景 2: 日常编码

- **推荐:** Cursor
- 实时 AI 辅助
- 原地编辑代码
- 无缝集成开发流程

最佳实践

组合使用:

- ① 学习阶段 → 通义千问
- ② 开发阶段 → Cursor
- ③ 重构优化 → Claude Code

真实场景

企业/学校要求：代码不能上传到外部服务器！

解决方案：本地部署

1. Ollama (推荐)

- 一行命令部署本地模型
- 支持 DeepSeek、Qwen 等
- 完全离线运行

2. LM Studio

- 图形界面，易于使用
- 支持多种模型格式
- 内置聊天界面

本地模型推荐

模型	特点
DeepSeek-Coder	编程专用
Qwen2.5-Coder	国产优秀
CodeLlama	Meta 开源
StarCoder2	多语言支持

Cursor 支持本地模型

Cursor 可以配置使用 Ollama 本地模型

不同模型编程能力对比

测试任务：用 Python 和 OpenCV 实现答题卡边界检测

模型	代码质量	中文理解	运行成功率	访问便利性
Claude 3.5	极好	很好	95%	需翻墙
GPT-4o	极好	很好	93%	需翻墙
通义千问 2.5	很好	极好	90%	国内直连
DeepSeek-Coder	很好	极好	88%	国内直连
GPT-3.5	一般	一般	75%	需翻墙

国际模型优势：

- 代码逻辑最严谨
- 边界情况处理完善
- 参数解释最详细

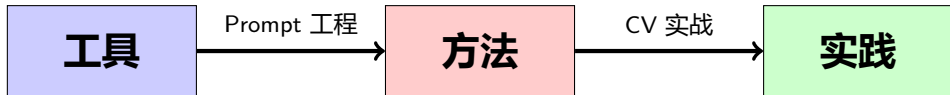
国产模型优势：

- 中文理解更自然
- 国内访问更稳定
- 价格更实惠/免费

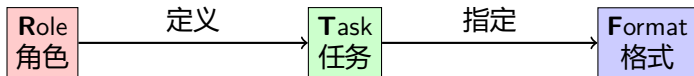
掌握了 Cursor、Claude Code 等工具后...

如何让 AI 听懂你的需求？

需要学习：Prompt 工程



结构化 Prompt 框架：RTF 模式



RTF 模板示例

Role (角色): 你是一个资深计算机视觉算法专家，精通 OpenCV 和图像处理。

Task (任务): 实现试卷图像的二值化，要求能自适应处理光照不均的情况。

Format (格式):

- 返回带有详细 Docstring 的 Python 函数
- 包含输入输出示例
- 列出关键参数的调优建议

[X] 不好的 Prompt

帮我写个代码处理图像。

问题分析：

- 没有定义角色
- 任务模糊
- 没有格式要求

[V] 好的 Prompt (RTF)

Role: 你是一位有 10 年经验的计算机视觉工程师，精通 OpenCV。

Task: 请编写一个 Python 函数，实现答题卡图像的自适应二值化。

Format:

- 提供完整的、可运行的代码
- 包含详细的函数文档
- 解释关键参数的设置依据

进阶技巧：Chain-of-Thought (思维链)

什么是思维链？

让 AI “**一步一步思考**” (Step by Step)，而不是直接给出答案。

[X] 直接给答案

Prompt: 用 OpenCV 实现透视变换矫正倾斜的试卷。

问题:

- AI 直接扔出代码
- 学生不理解原理
- 换个场景就不会了

[V] 思维链引导

Prompt:

请帮我实现试卷透视变换矫正。请按以下步骤思考：

Step 1: 透视变换的数学原理是什么？需要哪些参数？

Step 2: 如何从图像中自动找到试卷的四个角点？

Step 3: 请写出完整的 Python 实现代码

思维链实战：准确率对比

测试任务：用 OpenCV 实现自适应阈值处理

Prompt 类型	直接提问	思维链引导
代码完整度	70%	95%
参数解释清晰度	一般	详细
边界情况处理	很少提及	全面考虑
实际运行成功率	60%	90%

思维链 Prompt 模板

请帮我解决 [问题]。请按以下步骤思考：

Step 1: 分析问题的核心要点是什么？

Step 2: 有哪些可能的解决方案？各自的优缺点？

Step 3: 请给出推荐的实现代码

进阶技巧: Few-shot (少样本提示)

什么是 Few-shot?

给 AI 几个 **正确的例子**, 让它模仿你的风格或格式处理新任务。

示例场景: OpenCV 图像转换

待处理任务:

Few-shot Prompt

请模仿以下示例的代码风格和注释规范:

示例 1 - 灰度化:

```
# 读取图像并转换为灰度图
# 参数: image_path: 图像文件路径
# 返回: gray_image: 灰度图像数组
def load_and_gray(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)
    return gray
```

继续 Prompt

示例 2 - 高斯模糊:

```
# 对图像进行高斯模糊
# 参数: image: 输入图像
# 返回: blurred: 模糊后的图像
def gaussian_smooth(image):
    blurred = cv2.GaussianBlur(image, (5,5),
0)
    return blurred
```

任务:

请按照上述风格和规范, 实现一个图像边缘检测函数。

Few-shot 效果对比

Prompt	无 Few-shot	有 Few-shot
代码风格	随机, 不稳定	与示例高度一致
注释完整度	较简略	详细规范
参数说明	缺失	完整 Docstring
错误处理	经常遗漏	按示例添加
代码可读性	一般	优秀

核心技巧

- ① **示例数量：** 2-3 个示例通常足够
- ② **示例质量：** 确保示例是正确的、高质量的
- ③ **格式一致：** 示例之间保持风格一致
- ④ **明确指令：** 告诉 AI “请按照示例的风格”

常见错误

- 示例过多 (超过 5 个会增加混淆)
- 示例质量低 (AI 会学习坏习惯)
- 格式不统一 (AI 不知道该学哪个)

问题：AI 为什么会“忘”掉之前的内容？

上下文窗口有限，超出后最早的内容会被“遗忘”

各模型上下文限制：

模型	上下文
GPT-4	8K/32K
GPT-4o	128K
Claude 3.5	200K
DeepSeek	64K
通义千问	128K

精简 Prompt 技巧

- ① 只提供必要代码
- ② 使用摘要: "前面讨论了 X, 现在解决 Y"
- ③ 分段处理: 长任务拆分成多个短任务
- ④ 定期总结: 让 AI 总结当前进度

任务背景：试卷图像的噪声处理

故事问题

手机拍摄的试卷照片总是有噪点、光照不均, 如何自动处理这些问题?

常见图像质量问题:

- **噪声**: 手机传感器产生的随机噪点
- **光照不均**: 拍摄环境光线不均匀
- **模糊**: 手抖或对焦不准
- **透视变形**: 拍摄角度倾斜
- **阴影**: 遮挡造成的暗区

对阅卷系统的影响:

- 二值化效果差 → 填涂识别错误
- 边缘检测不准 → 答题卡定位失败
- 文字识别率低 → 主观题评分困难

本周目标

用 AI 辅助实现:

- ① 图像去噪 (高斯/中值滤波)
- ② 透视矫正 (透视变换)
- ③ 边缘检测优化

实战 1: 引导 AI 写出滤波代码

任务: 对比高斯滤波、中值滤波效果

第一轮 Prompt:

Role: 图像处理专家

Task: Python+OpenCV 实现去噪

要求: 高斯滤波、中值滤波对比

技巧: 多轮迭代、明确告诉 AI 需要改进什么

第二轮 Prompt:

添加 Docstring、解释适用场景、添加可视化、错误处理

实战 2：透视变换

挑战：透视变换数学原理复杂，如何通过 AI 快速实现？

第一轮：让 AI 解释原理

用简单比喻解释什么是透视变换

AI 比喻：

- 透视变换 = 把斜片"拉正"
- `getPerspectiveTransform` = 计算"怎么拉"
- `warpPerspective` = 执行"拉"

第二轮：实现代码

请根据上面的解释实现函数

关键步骤：预处理 → 找轮廓 → 定角点 → 透视变换

实战 3：自动识别答题卡边缘（迭代优化）

目标：通过 3 轮迭代 Prompt，让 AI 从写出简单 Canny 算子到写出鲁棒的轮廓提取逻辑。

第一轮：基础实现

基础 Prompt

请用 OpenCV 实现答题卡边缘检测。

AI 输出：

- 简单的 Canny 边缘检测
- 查找轮廓
- 可能问题：噪点干扰、参数固定

效果：较差

第二轮：优化改进

优化 Prompt

上面的代码检测效果不好，请改进：

- ① 先进行图像预处理（去噪）
- ② 使用自适应阈值
- ③ 筛选出最大的四边形

AI 输出：

- 高斯模糊去噪
- 自适应 Canny 阈值
- 面积筛选 + 四边形近似

效果：良好

实战 3 (续): 第三轮迭代

第三轮: 鲁棒完善

完善 Prompt

请进一步优化, 使其更鲁棒:

- ① 处理极端拍摄角度
- ② 处理光照不均和阴影
- ③ 添加透视变换矫正
- ④ 封装成可复用的类

AI 输出:

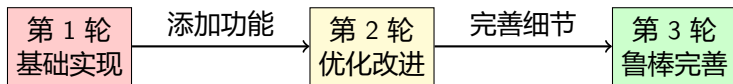
- 完整的类封装
- CLAHE 预处理
- 透视变换矫正
- 调试可视化

效果: **优秀**

迭代优化的关键

- **循序渐进**: 每轮只解决 1-2 个问题
- **具体反馈**: 告诉 AI 具体哪里不好
- **要求明确**: 提出具体的改进要求

迭代优化演示：从简单到完善



第 1 轮：基础

- 简单 Canny
- 查找轮廓
- 基本功能

问题：

- 噪点干扰
- 参数固定
- 不完整边缘

第 2 轮：优化

- 高斯模糊
- 自适应阈值
- 面积筛选

改进：

- 抗噪性增强
- 自动调参
- 鲁棒性提升

第 3 轮：完善

- 完整类封装
- CLAHE 预处理
- 透视变换

特性：

- 可复用性强
- 调试可视化
- 生产级可用

迭代优化的核心思想

为什么需要多轮迭代？

- **第 1 轮**：验证可行性，让代码“跑起来”
- **第 2 轮**：针对问题优化，让代码“好用”
- **第 3 轮**：完善细节，让代码“专业”

关键技巧

- 每轮只解决 1-2 个核心问题
- 不要一次性完成所有优化
- 每轮都要测试验证

与 AI 协作的迭代模式

- ① 第 1 轮 Prompt：实现基础功能
- ② 第 2 轮 Prompt：针对问题进行优化
- ③ 第 3 轮 Prompt：完善代码风格和注释

实战任务：用 AI 辅助实现人脸检测

Prompt 示例：

人脸检测 Prompt

请用 Python 和 OpenCV 实现一个人脸检测程序：

功能：从图片中检测所有人脸，并用矩形框标注

输入：图片文件路径

输出：标注了人脸框的图片

具体要求

- 使用 OpenCV 的 Haar 级联分类器
- 在每个人脸周围绘制绿色矩形框
- 显示检测到的人脸数量
- 代码有详细中文注释

完整的可运行代码，包含：

- ① 详细的函数文档（Docstring）
- ② 中文注释
- ③ 使用示例
- ④ 参数说明

提示

如果 AI 生成的代码有问题，可以：

- 把错误信息发给 AI，让它修复
- 询问 AI 如何优化代码
- 让 AI 解释代码的每一行

人脸检测代码示例

核心代码

```
def detect_faces(image_path): face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +  
'haarcascade_frontalface_default.xml') img = cv2.imread(image_path) gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) faces =  
    face_cascade.detectMultiScale(gray, 1.1, 5, (30, 30)) for (x, y, w, h) in faces :  
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2) return img
```

参数: scaleFactor=1.1, minNeighbors=5, minSize=(30,30)

(注: 参数需根据实际场景调整)

AI 辅助调试 (Debug) 三部曲

第 1 步

提供 Traceback

第 1 步：提供 Traceback

完整复制报错信息：

- 错误类型
- 错误位置 (行号)
- 完整的堆栈跟踪

示例：

```
cv2.error: ...  
Invalid number of channels
```

第 2 步

说明环境

第 2 步：说明环境

提供运行环境信息：

- 操作系统
- Python 版本
- OpenCV 版本
- 其他相关库版本

示例：

```
Windows 11  
Python 3.9.7  
OpenCV 4.8.0
```

第 3 步

贴出数据

第 3 步：贴出输入数据

提供输入数据信息：

- 数据类型和格式
- 数据的维度/大小
- 期望的输出格式
- 实际的输出 (如果有错误)

示例：

```
图像尺寸: 1920x1080  
通道: 3 (RGB)
```

完整的调试 Prompt 示例

我的 OpenCV 代码报错了，请帮我分析：

错误信息：

```
cv2.error: OpenCV(4.8.0) ...  
Invalid number of channels in input image
```

运行环境：

Windows 11, Python 3.9.7, OpenCV 4.8.0

相关代码：

```
img = cv2.imread('test.jpg')  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

输入数据：

图像文件 test.jpg, 尺寸 1920x1080

期望行为：正常转换为灰度图

重构：把 AI 当成导师

重构的意义

代码能运行只是第一步，**优雅的代码**才能长期维护。

重构 Prompt 模板：

重构请求

请帮我重构这段代码，提高代码质量：
要求：

- ① 提高运行效率
- ② 添加类型提示
- ③ 改进命名规范

AI 可能的重构建议：

- ① **性能优化**：向量化运算、避免重复计算
- ② **可读性**：变量名有意义、函数拆分合理
- ③ **健壮性**：输入校验、异常处理

我们强烈鼓励以下行为

① 设计高质量的 Prompt

- 使用 RTF 框架 (Role-Task-Format)
- 记录成功的 Prompt 模板
- 分享有效的 Prompt 给同学

② 用 AI 学习新的库和 API

- 请 AI 解释官方文档
- 请求代码示例和最佳实践
- 理解库的设计思想

学习提示

AI 是你的**学习伙伴**，而不是**代写工具**。善用 AI 可以提高学习效率，但不能替代你的思考过程。

更多鼓励的行为

继续鼓励的行为

① 用 AI 分析代码逻辑

- 代码审查，发现潜在 bug
- 分析算法复杂度
- 理解他人代码

② 记录学习过程

- 保存与 AI 的有价值对话
- 整理常见问题及解决方案
- 建立个人知识库

为什么要记录？

- 好的 Prompt 可以被复用
- 解决问题的思路可以积累
- 形成个人的知识体系

⚠ 重要提醒

使用 AI 辅助编程时，必须注意以下安全与伦理问题！

1. 数据安全

不要上传

- API Key、密码
- 个人隐私数据
- 商业机密代码

安全做法

- 使用脱敏示例数据
- 本地部署模型

2. 版权与责任

代码归属

AI 生成代码的版权复杂，检查所用工具的许可协议

责任界定

AI 生成的代码可能有错误，最终责任人是你自己

学术诚信要求

- 作业/论文中使用 AI 需声明
- 了解学校的 AI 使用政策
- 保持学习和思考的过程

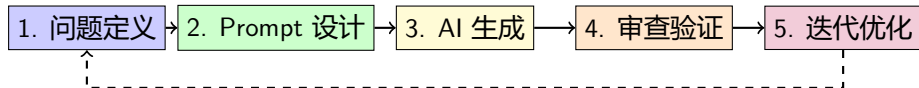
核心原则

- AI 是**辅助工具**，不是**替代品**
- 学习过程比结果更重要
- 理解代码原理是期末考试的重点

建立自己的“AI 协作 SOP”

什么是 SOP?

Standard Operating Procedure (标准操作流程)
一套可重复、可优化的工作流程



AI 协作 SOP：步骤详解

1. 问题定义

- 明确要解决的问题
- 确定输入输出
- 列出约束条件

2. Prompt 设计

- 应用 RTF 框架
- 使用思维链
- 提供 Few-shot 示例

3. AI 生成

- 运行 Prompt
- 获取初始输出
- 记录生成时间

4. 审查验证

- 检查代码逻辑
- 运行测试用例
- 验证输出结果

5. 迭代优化

- 识别改进点
- 设计优化 Prompt
- 重复流程直到满意

作业题目

用 AI 辅助实现 **答题卡边界检测**程序

项目关联

这是 **AI 阅卷助手**的第一步：

- ① 图像采集与预处理
- ② **答题卡定位（当前任务）**
- ③ 填涂检测与识别
- ④ 手写文字 OCR
- ⑤ 成绩统计与输出

差异化作业：三种难度可选

基础版

[观察者]

适合编程基础较弱的同学

- 运行 AI 生成的代码
- 观察不同参数效果
- 完成基础报告

要求：

- AI 对话至少 2 轮
- 检测标准答题卡
- 基础功能报告

进阶版

[使用者]

适合有一定基础的同学

- 调整参数优化效果
- 处理不同场景
- 完成完整报告

要求：

- AI 对话至少 3 轮
- 处理倾斜/暗光场景
- 包含参数分析

挑战版

[创造者]

适合基础较好的同学

- 实现自动检测
- 处理复杂背景
- 完成深度分析

要求：

- AI 对话至少 4 轮
- 自动找答题卡边界
- 复杂场景鲁棒性

① AI 对话记录 (必交)

- 截图或复制文本
- 展示完整交互过程
- 标注关键 Prompt 设计

② 最终代码 (必交)

- 完整可运行代码
- 包含必要注释
- 标注 AI 辅助部分

③ 测试结果 (必交)

- 测试用例图片
- 处理结果图片
- 标注检测边界

④ 反思报告 (必交)

- Prompt 设计思路
- 问题及解决过程
- AI 辅助编程思考

评分表

评分项	分值
AI 对话记录	25 分
Prompt 设计质量	25 分
代码完整性	20 分
测试结果	15 分
反思报告深度	15 分
总分	100 分

难度系数

- 基础版：系数 1.0（最高 100 分）
- 进阶版：系数 1.1（最高 110 分）
- 挑战版：系数 1.2（最高 120 分）

评分重点

- 过程 > 结果
- Prompt 设计能力
- 反思与思考

谢谢!

第 3 周预告：图像预处理与增强

故事问题：试卷拍照模糊怎么办？

- 图像去噪（高斯/中值滤波）
- 图像二值化（全局/Otsu/自适应）
- 透视矫正（透视变换）